Dokument etapu 2: *Architektura systemu i projekt techniczny*

Tytuł projektu: *Projekt i implementacja platformy testowej weryfikującej i profilującej rozwiązania programistyczne*

Autorzy: Kacper Grobel, Kamil Błażejczyk, Kornel Dzieża
Promotorzy: mgr inż. Tomasz Herman, dr inż. Michał Dębski



Wydział Matematyki i Nauk Informacyjnych
POLITECHNIKA WARSZAWSKA

4 listopada 2026

Abstrakt

Niniejszy dokument przedstawia architekturę platformy testowej przeznaczonej do zautomatyzowanej weryfikacji i profilowania studenckich rozwiązań programistycznych. Głównym celem opracowania jest zdefiniowanie struktury modułowej powstającego rozwiązania, określenie odpowiedzialności poszczególnych komponentów oraz opisanie interfejsów i warstw komunikacyjnych pomiędzy nimi. Ponadto dokument zawiera graficzne schematy przepływu sterowania, wizję punktów styku z użytkownikiem oraz merytoryczne uzasadnienie wyboru stosu technologicznego.

Historia zmian dokumentu

Wersja	Data	Autorzy	Opis zmian
1.0	03.11.2026	KG / KB / KD	Wersja bazowa dokumentu
1.1	04.11.2026	KG	Zmiana opisu mechanizmu komunikacji i przepływu danych pomiędzy modułami CIL Injection i Test Executor

Spis treści

1 Projekt i architektura systemu **3**
1.1 Architektura systemu – moduły i ich odpowiedzialności ... 4
1.1.1 Izolowane środowisko uruchomieniowe ... 4
1.1.2 Moduł modyfikacji kodu pośredniego CIL (biblioteka CilInjection) ... 4
1.1.3 Moduł koordynacji testów (Test Executor) ... 5
1.2 Projekt modułów – interfejsy i zależności ... 5
1.3 Projekt warstwy komunikacyjnej ... 6
1.4 Schematy głównych komponentów ... 7
1.5 Wizja interfejsu użytkownika – schematy i rysunki ... 12
2 Uzasadnienie wyboru stosu technologicznego **13**
3 Specyfikacja interfejsu fasady testowej **14**
3.1 Kluczowe komponenty fasady ... 14
3.2 Automatyzacja i generowanie kodu zestawów testowych ... 15
3.2.1 Filtrowanie i analiza składniowa ... 15
3.2.2 Generowanie kodu wyjściowego ... 15
3.3 Przepływ tworzenia zadań przez prowadzącego ... 16

Słowniczek pojęć

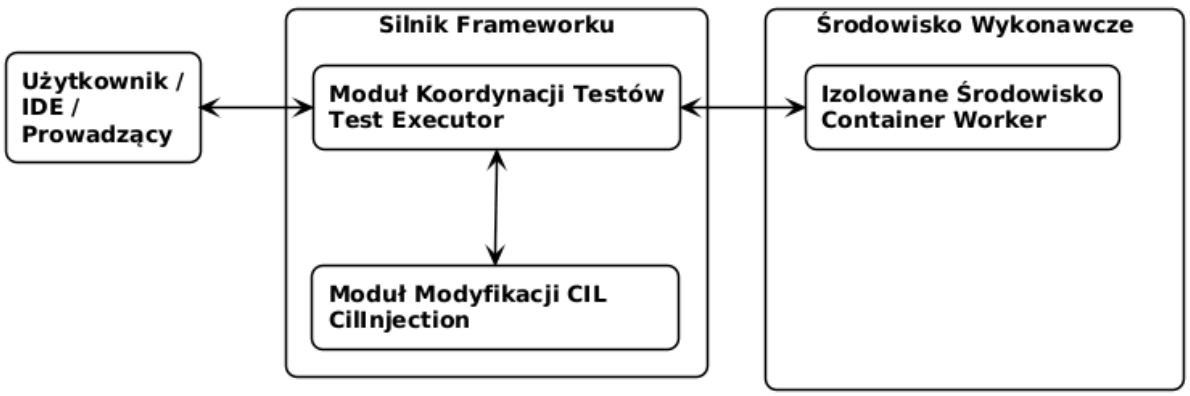
Pojęcie	Definicja
AAA (*Arrange-Act-Assert*)	Wzorzec projektowania testów dzielący je na trzy fazy: przygotowanie środowiska (Arrange), wykonanie akcji (Act) oraz weryfikację wyników (Assert).
Analiza statyczna	Badanie kodu źródłowego (np. jego drzewa składniowego) przed uruchomieniem, używane w systemie m.in. do weryfikacji użycia niedozwolonych konstrukcji.
ASD2 (*Algorytmy i Struktury Danych 2*)	Przedmiot, dla którego jest przeznaczony ten projekt i na którego potrzeby powstaje platforma.
Asercja (*Assert*)	Instrukcja w teście weryfikująca, czy uzyskany wynik lub stan (np. spójność grafu) jest zgodny z oczekiwaniami.
Assembly / Assembly Bytes	Skompilowany plik biblioteki (np. `.dll`) lub jego reprezentacja w pamięci w postaci tablicy bajtów, poddawana modyfikacjom i uruchamiana w kontenerze.
C#	Obiektowy język programowania ogólnego przeznaczenia kompilowany do kodu CIL. Język, w którym studenci piszą rozwiązania.
CI/CD (*Continuous Integration / Deployment*)	Praktyka automatyzacji budowania, testowania i wdrażania oprogramowania przy każdej zmianie kodu. Środowisko wykonujące te czynności nazywa się środowiskiem CI/CD.
CIL (*Common Intermediate Language*)	Język pośredni, do którego kompiluje się kod C# przed ostatecznym wykonaniem przez środowisko uruchomieniowe .NET.
CLI (*Command Line Interface*)	Interfejs tekstowy, w którym programem steruje się i w którym prezentowane są wyniki w postaci logów lub tabel.
Docker	Narzędzie do konteneryzacji aplikacji, pozwalające na tworzenie powtarzalnych i bezpiecznych środowisk.
Drzewo składniowe (*Syntax Tree*)	Hierarchiczna struktura danych reprezentująca kod źródłowy programu po jego przetworzeniu przez kompilator.
Fasada (Fasada testowa)	Wzorzec projektowy ukrywający skomplikowaną logikę uruchamiania testów za prostym w użyciu interfejsem deklaratywnym przeznaczonym dla prowadzącego.
GitHub Actions	System CI/CD serwisu GitHub, w którym środowisko i procesy automatycznej weryfikacji są zdefiniowane jako tzw. akcje.
IDE (*Integrated Development Environment*)	Zintegrowane środowisko programistyczne (np. JetBrains Rider, Visual Studio) służące do tworzenia i lokalnego testowania kodu.
Kontener / konteneryzacja	Kontener to izolowane środowisko uruchomieniowe aplikacji współdzielące jądro systemu gospodarza, ale blokujące dostęp do jego zasobów (np. sieci). Konteneryzacja to proces używania takich środowisk.
Model semantyczny	Struktura w kompilatorze Roslyn dostarczająca informacji o znaczeniu elementów kodu, wykorzystywana do odczytywania atrybutów i typów.
MSTest	Wbudowany w platformę Microsoft framework do testów jednostkowych, którego mechanizmy są tu użyte do integracji wyników weryfikacji z interfejsem narzędzi w IDE.
.NET 10	Wersja platformy programistycznej firmy Microsoft. W jej skład wchodzi środowisko uruchomieniowe wykonujące kod CIL i izolujące zarządzanie pamięcią.

Pojęcie	Definicja
NuGet	Narzędzie oraz ekosystem zarządzania pakietami (bibliotekami) dla platformy .NET, służące jako metoda dystrybucji systemu do użytkowników.
Orkiestracja / Orkiestrator	Automatyczne zarządzanie, układanie w sekwencję i koordynowanie zadań pomiędzy niezależnymi modułami (np. zadanie modułu Test Executor).
Przestrzeń nazw (*Namespace*)	Mechanizm C# grupujący typy i organizujący ich nazwy (np. `System.IO`).
Refleksja (*Reflection*)	Mechanizm języka C# pozwalający programowi na dynamiczne skanowanie swoich struktur (np. wyszukiwanie klas testowych lub metod) w trakcie działania aplikacji.
Roslyn	Platforma kompilatora .NET używana w systemie do analizy statycznej, dynamicznej kompilacji w pamięci oraz przyrostowego generowania kodu w locie.
Sprawdzarka	Potoczne określenie zautomatyzowanego systemu oceniającego rozwiązania studentów.
`stdout` (*Standard Output*)	Standardowy strumień wyjścia procesu, na który system wypisuje rezultaty testów w potoku automatyzacji procesów (CI/CD).
Test Executor	Komponent orkiestrujący: analizuje kod, deleguje wplatanie instrukcji CIL, osadza pliki w kontenerze, uruchamia proces i zwraca wyniki.
Wstrzykiwanie / Instrumentacja CIL	Modyfikacja skompilowanego zestawu przez dodanie instrukcji języka pośredniego (np. liczników operacji) bez konieczności jakiejkolwiek ingerencji w kod źródłowy.
Mono.Cecil	Biblioteka do odczytu i modyfikacji kodu CIL, używana do wplatania instrukcji.

1 Projekt i architektura systemu

Niniejszy rozdział szczegółowo przedstawia architekturę oraz projekt techniczny zautomatyzowanej platformy testowej. Prezentuje on ogólny podział systemu na niezależne, lecz współpracujące ze sobą moduły, jednocześnie określając ich granice i obowiązki podczas weryfikacji oraz profilowania kodu.

Ogólny zarys zaprojektowanej architektury, obrazujący hierarchię oraz relacje pomiędzy najważniejszymi modułami rozwiązania, został zilustrowany na rysunku 1.



Rysunek 1: Diagram modułów systemu.

1.1 Architektura systemu – moduły i ich odpowiedzialności

Architektura platformy została zaprojektowana na podstawie wyspecjalizowanych, słabo powiązanych modułów. Każdy z nich realizuje ściśle określony zakres odpowiedzialności. Główny przepływ sterowania oraz obieg danych w systemie opiera się na współpracy trzech kluczowych komponentów:

1. Moduł koordynacji testów (Test Executor) – stanowiące centralny punkt orkiestracji procesu testowego.
2. Moduł modyfikacji kodu pośredniego (CilInjection) – odpowiedzialne za niskopoziomową instrumentację kodu CIL i zliczanie instrukcji.
3. Izolowane środowisko uruchomieniowe – zapewniające bezpieczne i odseparowane wykonanie testów wewnątrz kontenerów.

1.1.1 Izolowane środowisko uruchomieniowe

Jednym z podstawowych wymagań biznesowych dla nowego systemu jest izolacja rozwiązań studentów. Ma ona na celu zarówno wykluczenie możliwości wyrządzenia szkód przez złośliwy kod, który może być wysłany do systemu, jak i zapewnienie czystego środowiska dla każdej kolejnej instancji testującej kody studentów.

Komponentem realizującym to wymaganie jest izolowane środowisko uruchomieniowe, czyli konteneryzowany program uruchamiający przygotowany wcześniej zestaw testów. Rozwiązanie to ogranicza możliwość oddziaływania wykonywanego kodu na systemem gospodarza, a ponadto daje pełną kontrolę nad działającym środowiskiem. W przypadku, gdy program testujący przestanie odpowiadać albo ulegnie zapętleniu (np. z powodu błędu logicznego w kodzie studenta), możliwe jest wymuszone zatrzymanie kontenera za pomocą polecenia `docker kill`. Dodatkowo, proces ten można zintegrować z mechanizmem limitu czasu, definiując maksymalny czas wykonania, po przekroczeniu którego środowisko zostanie automatycznie zamknięte.

Zakres odpowiedzialności tego modułu jest następujący:

- wczytanie otrzymanego zestawu testów,
- przeprowadzenie testów,
- zapisanie wyniku każdego przeprowadzonego testu do pliku.

1.1.2 Moduł modyfikacji kodu pośredniego CIL (biblioteka CilInjection)

Całkowicie wewnętrzny komponent systemu, wydzielony w postaci niezależnej biblioteki, odpowiadający za niskopoziomową manipulację skompilowanym kodem CIL za pomocą bibioteki Mono.Cecil.

Zadania modułu podzielone są na dwa wewnętrzne obszary funkcjonalne:

- Definiowanie reguł i rozbudowa systemu (dla deweloperów utrzymujących framework):
  - Udostępnianie ustandaryzowanych struktur i klas bazowych, pozwalających programistom na łatwe dodawanie nowych, wewnętrznych mechanizmów transformacji kodu IL, bez konieczności ingerencji w główny kod silnika wykonawczego.
  - Dostarczanie abstrakcji ukrywających złożoność manipulacji surowymi instrukcjami języka pośredniego.
- Mechanizm wykonawczy (wykorzystywany przez silnik frameworku):
  - Zarządzanie potokiem modyfikacji – przyjmowanie skompilowanych plików (*Assembly Bytes*) testowanego kodu, sekwencyjne aplikowanie wymaganych transformacji i generowanie zmodyfikowanych plików (*Assembly Bytes*).
  - Zapewnienie mechanizmów do bezpiecznego, dynamicznego ładowania zmodyfikowanego kodu w izolowanym kontekście pamięci (`AssemblyLoadContext`).
  - Zapewnienie mechanizmu do bezpiecznego, dynamicznego ładowania wymaganych przez wprowadzone modyfikacje wewnętrznych bibliotek do izolowanego kontekstu pamięci (`AssemblyLoadContext`).
  - Wygenerowanie wymaganych przez zmiany w kodzie studenta bibliotek pomocniczych (*Assembly Bytes*).
  - Zapewnienie pełnych ścieżek do metod do obsługi bibliotek pomocniczych, w celu umożliwienia późniejszego ich wywołania za pomocą refleksji.

1.1.3 Moduł koordynacji testów (Test Executor)

Najważniejszym komponentem łączącym wszystkie pozostałe w całość jest moduł koordynatora. Odpowiada on za kompleksowe przeprowadzenie testów rozwiązania studenta. W trakcie swojej pracy korzysta on ze wszystkich wyżej opisanych komponentów, przygotowując im dane do przekazania, odbierając wyniki ich pracy i przekazując je dalej. Właśnie ten moduł definiuje kontrakt, za pomocą którego odbywa się komunikacja z użytkownikiem systemu.

Oprócz głównej funkcji delegowania zadań, moduł Test Executor realizuje również operacje pomocnicze, których złożoność nie uzasadnia wyodrębnienia ich do niezależnego komponentu. Należą do nich:

- analiza statyczna kodu – wykonywana w celu wczesnego odrzucenia rozwiązań wykorzystujących niedozwolone przestrzenie nazw:
  - `System.IO`,
  - `System.Reflection`,
  - `System.Diagnostics`,
  - `System.Threading`.
- kompilacja kodu.

Dokładny zakres odpowiedzialności koordynatora wynika z kształtu procesu uruchamiania testów i zwracania wyników do użytkownika i wygląda następująco:

- interpretacja testów zdefiniowanych przez użytkownika,
- zarządzanie testami w sposób przyjazny dla IDE (obsługa współbieżności),
- uruchomienie i obsługa potoku testowego:
  - analiza statyczna,
  - dynamiczna kompilacja do pamięci,
  - wstrzyknięcie kodu CIL zliczającego instrukcje (z wykorzystaniem biblioteki opisanej w rozdziale 1.1.2),
  - przygotowanie środowiska opisanego w rozdziale 1.1.1 i uruchomienie w nim testów rozwiązania studenta,
  - oczekiwanie na wyniki spływające z wyżej wymienionego środowiska z uwzględnieniem możliwości wystąpienia niepowodzeń,
  - przekazanie wyników do użytkownika.

1.2 Projekt modułów – interfejsy i zależności

W tym rozdziale dokładniej opisane zostaną kontrakty i hierarchia zależności pomiędzy zaprezentowanymi wcześniej modułami. Opisany zostanie także graf zależności, widoczny na rysunku 1.

Jak wspomniano w rozdziale 1.1.3, moduł koordynacji testów jest elementem, który łączy pozostałe komponenty w całość. Z tego powodu jako wejściową część jego kontraktu należy przyjąć strukturę otrzymywaną poprzez fasadę, która zostanie kompleksowo opisana w rozdziale 3. Jako wyjście, moduł ten zwraca wyniki testów, do których można się dostać poprzez współbieżnie bezpieczną funkcję (*getter*). Z punktu widzenia użytkownika końcowego, funkcja ta będzie wywoływana asynchronicznie przez kod automatycznie wygenerowany na podstawie zdefiniowanego wcześniej zestawu testów. W ten sposób każda interakcja z systemem możliwa będzie za pośrednictwem narzędzia uruchamiania testów w ramach użytkowanego IDE.

Podczas swojej pracy, moduł koordynujący będzie zależny zarówno od biblioteki wstrzykującej instrukcje CIL, jak i izolowanego środowiska uruchomieniowego, które są wyspecjalizowanymi komponentami realizującymi konkretne zadania. Z tego powodu w obu przypadkach kontrakt jest minimalny.

Moduł wstrzykujący kod CIL jako wejście otrzymuje kod studenta jako skompilowany plik (*Assembly Bytes*). Skompilowany kod studenta jest zmieniany poprzez dodanie sekwencji instrukcji zwiększających licznik. Sekwencja ta jest dodawana między każdą instrukcję CIL. Licznik znajduje się w osobnej bibliotece zawierającej pole *Counter*. Wyjściem tego modułu są nowy skompilowany plik kodu studenta (*Assembly Bytes*), skompilowany plik biblioteki z polem *Counter* (*Assembly Bytes*) oraz pełne ścieżki do metod z biblioteki z polem *Counter*. Moduł ten jest zawsze uruchamiany przez Test Executor.

Izolowane środowisko uruchomieniowe jako wejście otrzymuje gotową do uruchomienia bibliotekę (`.dll`), zawierającą zestaw testów. Każdy z testów jest wykrywany za pomocą refleksji i uruchamiany sekwencyjnie. bibliotekę (`.dll`) zawierającą kod studenta, bibliotekę (`.dll`) z polem *Counter* oraz pełne ścieżki do metod z biblioteki z polem *Counter*. Metody do biblioteki z polem *Counter* są wykrywane

poprzez refleksje i wywoływane w celu dostania lub zresetowania licznika. Wyjściem tego modułu jest zestaw wyników wszystkich testów, zapisywany do plików. W kontekście całego systemu moduł ten zawsze jest uruchamiany przez Test Executor, który ma również możliwość przeglądania plików wewnątrz kontenera. Z tego powodu ma on dostęp do informacji o wynikach testów na bieżąco, gdyż sprawdzenie tego faktu to wyszukanie odpowiedniego pliku wewnątrz kontenera. Schemat komunikacji pomiędzy izolowanym środowiskiem uruchomieniowym a modułem koordynującym zostanie dokładniej opisany w rozdziale 1.3.

1.3 Projekt warstwy komunikacyjnej

Z faktu, że system jest w znakomitej większości planowany jako jedna biblioteka, większość komunikacji odbywa się wewnątrzprocesowo, w ramach jednego środowiska uruchomieniowego .NET. Jednak w ramach systemu zdarzają się również inne sposoby komunikacji. W celu zilustrowania ich roli i sposobu realizacji, poniżej przeanalizowano uproszczony cykl życia kodu nadesłanego przez studenta.

Na początek student wypycha zmiany swojego kodu w dedykowanym repozytorium bądź uruchamia system lokalnie. Kod źródłowy trafia wtedy bezpośrednio do modułu koordynacji testów, czyli Test Executor.

Następnym krokiem jest analiza i kompilacja rozwiązania, a następnie przekazanie skompilowanych bajtów do modułu modyfikacji kodu pośredniego. Zwróćmy uwagę, że odbywa się to wewnątrz tego procesu.

Po zakończeniu pracy przez bibliotekę wstrzykującą kod CIL koordynator odbiera zaktualizowane bajty programu, a następnie przekazuje je wraz z regułami pomiarowymi do izolowanego środowiska uruchomieniowego. Ta operacja jest zaprojektowana jako osadzenie pliku `.dll` wewnątrz kontenera przez moduł koordynatora. Taki osadzony plik jest przez program uruchamiający widziany jak zwykły plik w systemie plików.

Do uruchomienia izolowanego środowiska uruchomieniowego wykorzystywana jest biblioteka `TestContainers`. Pozwala ona pobranie kontenera z repozytorium (np. DockerHub), a następnie wysokopoziomowe uruchomienie za pomocą jednego wywołania. Dodatkową zaletą użycia tej biblioteki jest automatyczne wyłączenie kontenera i sprzątanie po nim po zakończeniu jego pracy.

Izolowane środowisko uruchomieniowe uruchamia otrzymane rozwiązanie studenta poprzez odczytanie osadzonego pliku i jego interpretację jako biblioteka. Dzięki temu system macierzysty pozostaje bezpieczny, a zasoby są limitowane. Każdy z testów jest sekwencyjnie wykonywany, a jego wynik zostaje natychmiast zapisywany do pliku w formacie JSON. Mimo że aplikacja docelowa traktuje to jako standardową operację wejścia-wyjścia na nośniku, faktyczny zapis odbywa się w izolowanym systemie plików kontenera. Ta metoda komunikacji została wybrana przede wszystkim ze względu na jej prostotę oraz możliwość całkowitej izolacji kontenera. Sieć ani żaden inny zasób zewnętrzny nie jest potrzebny w żadnym momencie wykonywania kodu – można więc bezpiecznie pozbawić program uruchamiany wewnątrz kontenera dostępu do nich.

Test Executor, dzięki możliwości odczytywania zawartości kontenera, może sprawdzać obecność plików na jego wirtualnym dysku. Obecność pliku odpowiadającego danemu testowi jest jednoznaczna z zakończeniem jego wykonywania. Może to być zarówno test zakończony powodzeniem, jak i porażką. Sytuację zawieszenia się kontenera bądź przedwczesnego zakończenia jego pracy można wykryć poprzez ustalenie limitu czasu dla całego zestawu testów i jego kontrolę.

Agregacja i zwrot wyników testów planowany jest na dwa sposoby, w zależności od sposobu uruchomienia. W przypadku lokalnego uruchomienia z poziomu narzędzia do uruchamiania testów w IDE, wyniki będą prezentowane jako stan końcowy testów we wspomnianym narzędziu. Zakończenie testu z wynikiem pozytywnym jest wizualizowane w IDE kolorem zielonym. W przypadku negatywnej weryfikacji wynik oznaczany jest kolorem czerwonym, a system dostarcza dodatkowe dane o charakterze zaistniałego defektu. Natomiast instancje uruchamiane w ramach CI/CD na repozytoriach (np. w serwisie GitHub) będą wyświetlały wyniki jako tekst w logu danej akcji, a ponadto będą wysyłały te wyniki do dedykowanego serwera bądź repozytorium – katalogu wyników studentów – w celu dalszego użytkowania tych danych przez prowadzącego i studentów.

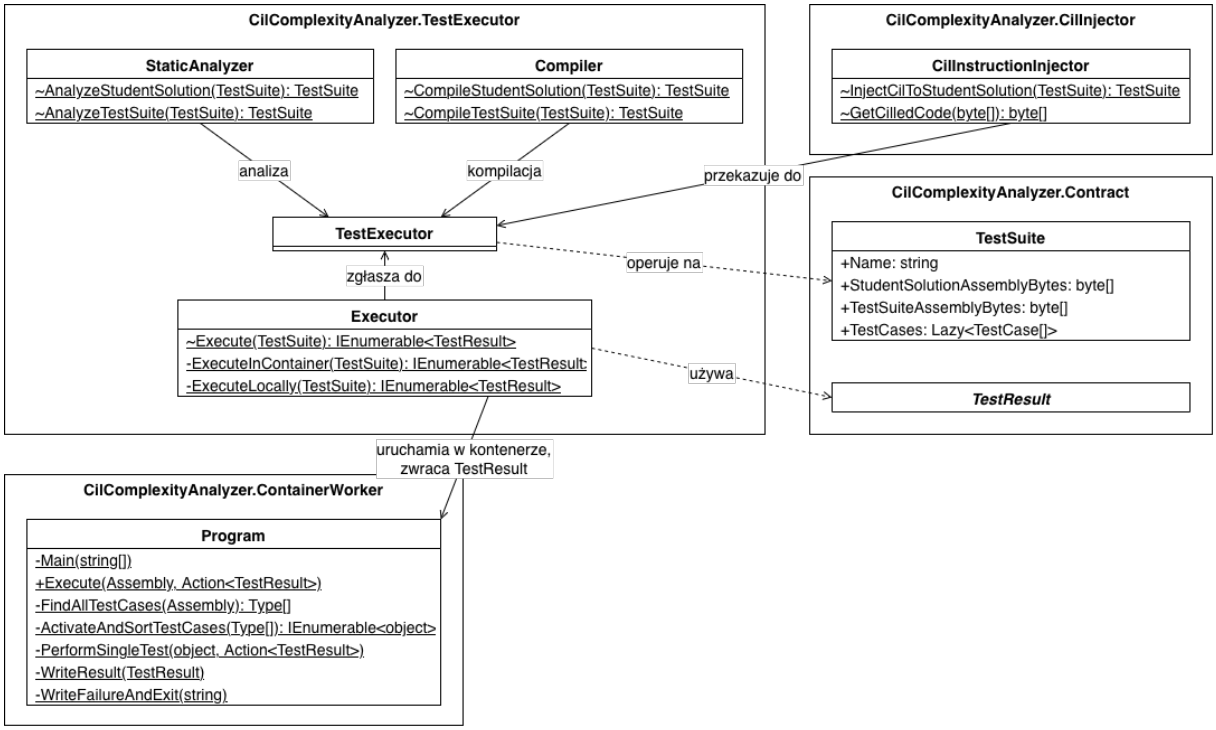
1.4 Schematy głównych komponentów

Architektura modułu opiera się na jednej klasie koordynującej, `TestExecutor`, oraz wyspecjalizowanych komponentach realizujących kolejne etapy przetwarzania. `TestExecutor` sam nie wykonuje pracy merytorycznej, tylko sekwencjonuje etapy i zarządza ich wynikami.

Etapy przygotowania kodu (`StaticAnalyzer`, `Compiler` oraz `CilInstructionInjector`) są bezstanowymi klasami statycznymi. Każda metoda przyjmuje `TestSuite` i zwraca go wzbogaconego o kolejne wyniki, więc przetwarzanie ma postać potoku, a etapy są niezależne i łatwe do testowania w izolacji.

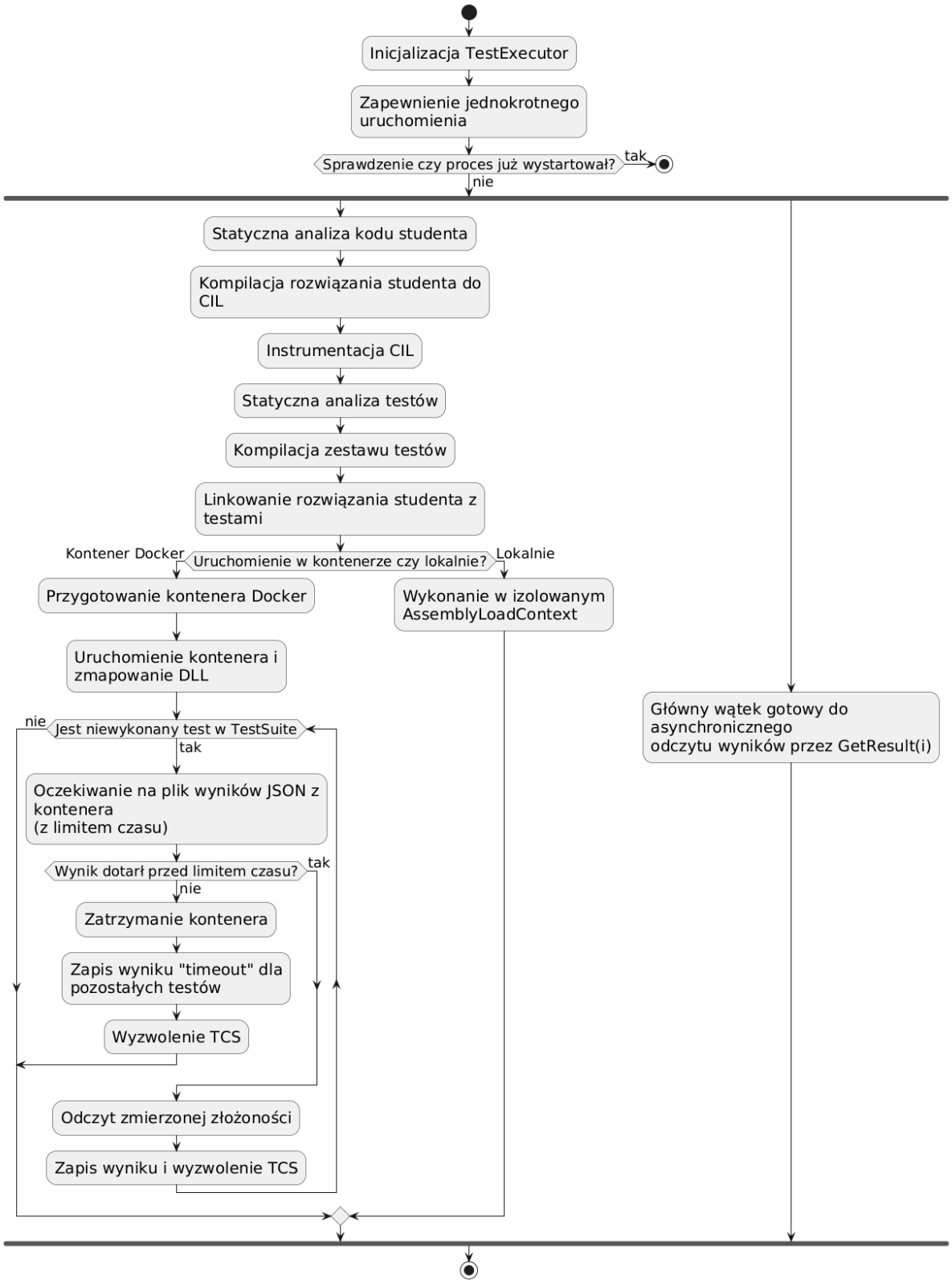
Wykonanie testów jest wydzielone do klasy `Executor`, która ukrywa za jednym kontraktem dwa tryby uruchomienia: lokalny i w kontenerze. Wyniki są zwracane jako `IEnumerable<TestResult>`, czyli strumieniowo, w miarę wykonywania testów.

Tryb kontenerowy obsługuje osobny projekt `ContainerWorker` z własnym punktem wejścia. Izoluje on niezaufany kod studenta od procesu frameworka i ogranicza zależności do niezbędnego minimum. Komunikację między modułami zapewnia pakiet `Contract` z typami `TestSuite` i `TestResult`, dzięki czemu projekty nie referencjonują się nawzajem i nie powstają cykle zależności.



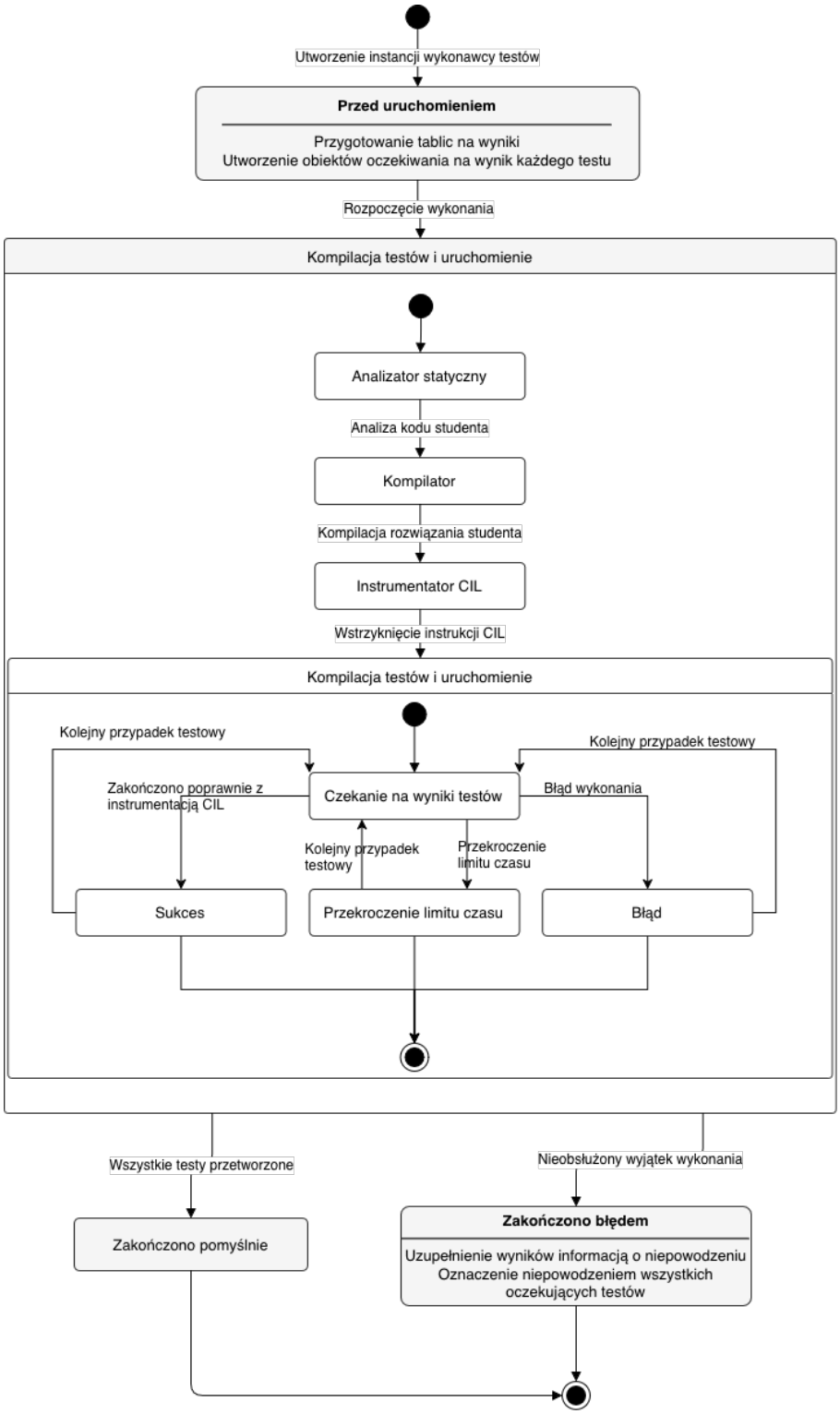
Rysunek 2: Diagram klas modułu TestExecutor oraz powiązanych komponentów potoku.

Wykres aktywności widoczny na rysunku 3 prezentuje współbieżny proces przygotowania logiki testów oraz środowiska uruchomieniowego. Zobrazowany jest tutaj także mechanizm kontroli czasu wykonania.



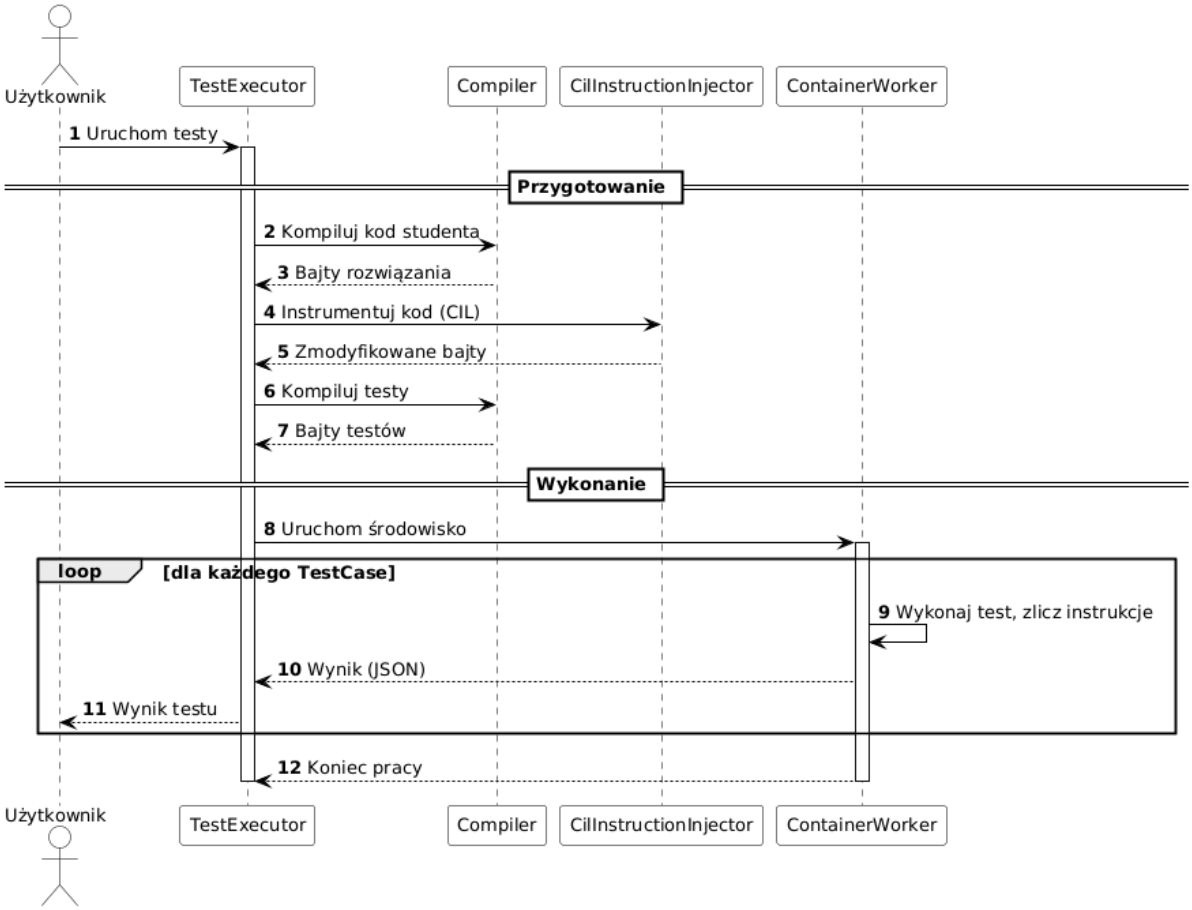
Rysunek 3: Diagram aktywności przedstawiający proces kompilacji, wstrzykiwania kodu CIL oraz wykonania testów.

Diagram widoczny na rysunku 4 obrazuje cykl życia głównego obiektu sterującego od jego inicjalizacji do ostatecznego zakończenia sukcesem lub porażką. Szczegółowo rozpisuje na mniejsze podetapy kompilacji i iteracyjnego uruchamiania testów, uwzględniając obsługę wyjątków i przekroczeń limitu czasu.



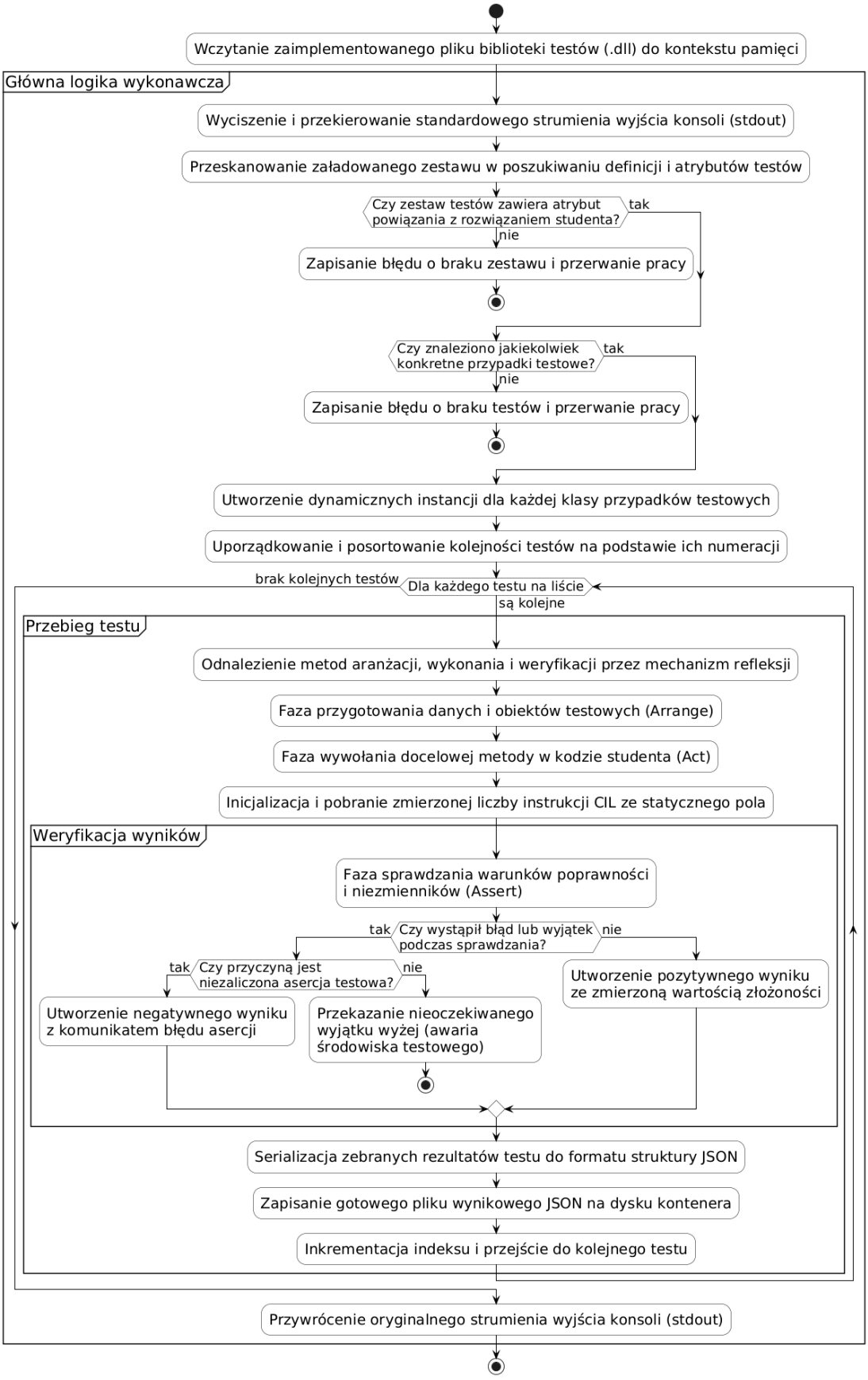
Rysunek 4: Diagram stanów cyklu życia orkiestracji i wykonania testów w module TestExecutor.

Wykres sekwencji widoczny na rysunku 5 modeluje wymianę komunikatów w czasie między interfejsem użytkownika, koordynatorem testów, narzędziami modyfikującymi kod oraz izolowanym środowiskiem uruchomieniowym. Obrazuje sekwencyjne delegowanie zadań kompilacji, wstrzykiwania kodu CIL oraz pętlę odpytywania kontenera o rezultaty wykonania poszczególnych przypadków testowych.



Rysunek 5: Diagram sekwencji przedstawiający przepływ sterowania, kompilację, wstrzykiwanie kodu CIL oraz sekwencyjne uruchamianie testów w środowisku izolowanym.

Diagram aktywności zaprezentowany na rysunku 6 szczegółowo opisuje procesy zachodzące bezpośrednio wewnątrz uruchomionego kontenera testowego. Przedstawia krok po kroku wyszukiwanie klas testowych przez refleksję, izolację wyjścia konsoli oraz bezpieczne wywoływanie funkcji *Arrange*, *Act* i *Assert*, zakończone serializacją obiektów TestResult do plików JSON.



Rysunek 6: Diagram aktywności dla kontenera testowego.

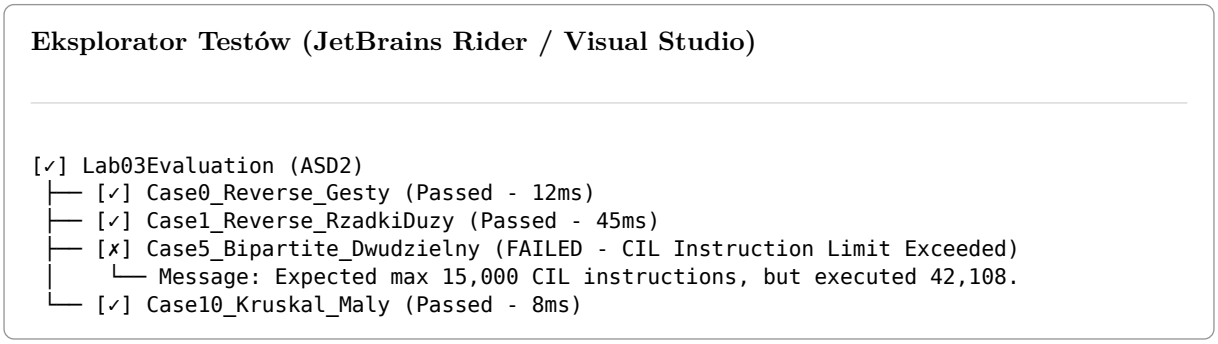
1.5 Wizja interfejsu użytkownika – schematy i rysunki

Interfejs platformy został pomyślany tak, by łączyć dwa główne punkty styku z użytkownikiem. Z jednej strony jest to interfejs środowiska programistycznego (IDE), służący do pracy lokalnej, z drugiej zaś interfejs wiersza poleceń (CLI), a także raporty CI/CD, które uruchamiane są w środowisku automatycznej oceny.

W środowisku programistycznym, na przykład w JetBrains Rider czy Visual Studio, testy zdefiniowane w ramach platformy są wykrywane automatycznie. Pojawiają się one w natywnym oknie uruchamiania testów, takim jak Unit Tests Explorer. To możliwe dzięki mechanizmom opartym pod spodem na interfejsie MSTest.

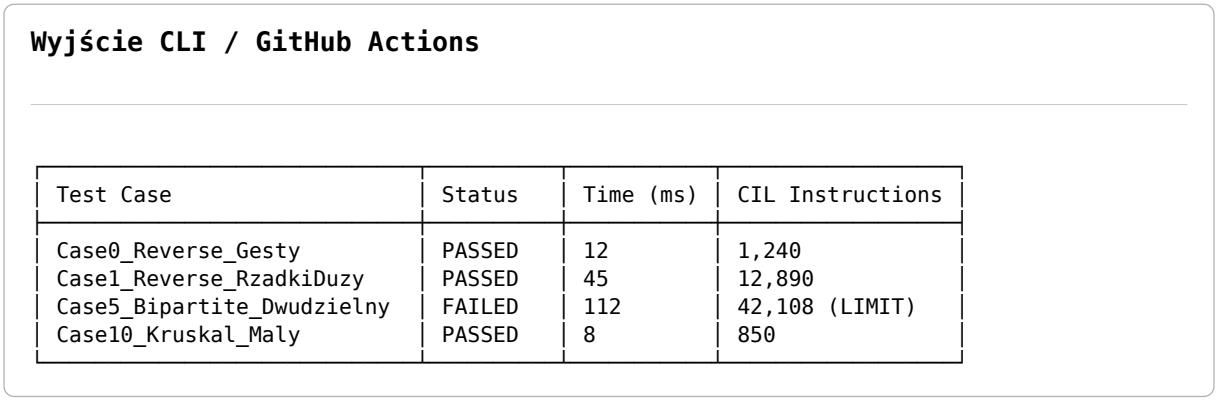
1. Widok eksploratora testów: Każdy przypadek testowy (`TestCase`) trafia do eksploratora jako osobny test jednostkowy, przypisany do konkretnej kategorii bądź zadania.
2. Prezentacja wyników i asercji: Jeśli test zakończy się niepowodzeniem, na przykład z powodu naruszenia niezmiennika grafu albo przekroczenia dopuszczalnej wartości skompilowanych instrukcji CIL, panel szczegółów pokazuje czytelny komunikat o błędzie. Opisuje on, czego oczekiwano, a co otrzymano.

Interfejs użytkownika, jako że platforma jest narzędziem wspierającym proces wytwarzania oprogramowania, nie opiera się na klasycznym interfejsie graficznym (GUI). Punktem styku z użytkownikiem (studentem i prowadzącym) jest środowisko zintegrowane (IDE) za pośrednictwem eksploratora testów, a także konsola (CLI) w zautomatyzowanym środowisku CI/CD.



Rysunek 7: Szkic wizualizacji wyników testu w oknie eksploratora testów w środowisku IDE.

W oknie testów IDE prowadzący i student otrzymują natychmiastową informację zwrotną w postaci hierarchicznego drzewa testów. Statusy sukcesu lub błędu są powiązane bezpośrednio z wynikami wyliczonymi przez platformę testową (m.in. weryfikacją liczby wykonanych instrukcji CIL).



Rysunek 8: Szkic raportu podsumowującego wyniki i wskaźniki złożoności w konsoli CI/CD.

W potoku automatyzacji procesów (CI/CD) system wypisuje rezultaty w standardowym wyjściu procesu (stdout). Dane będą mogły być wyświetlone w postaci tabelarycznej, udostępniając agregację pomiarów wydajności i kosztów wyrażonych w instrukcjach języka pośredniego, jak pokazuje przykład powyżej.

2 Uzasadnienie wyboru stosu technologicznego

Tabela 2 przedstawia zestawienie kluczowych komponentów architektonicznych oraz narzędzi wybranych oraz ich implementacji. Każda decyzja technologiczna została podjęta na podstawie analizy wymagań stawianych przez klienta, preferencji dotyczących dalszego utrzymania systemu oraz doświadczeń zespołu. Uwzględniono w niej również bilans zysków i strat w stosunku do rozważanych alternatyw.

Tabela 2: Zestawienie wybranych technologii dla kluczowych modułów systemu wraz z uzasadnieniami i odrzuconymi alternatywami.

Moduł	Wybrana technologia	Możliwe alternatywy	Uzasadnienie
Backend (moduł uruchamiający)	C# .NET 10, Docker	Rust, C++	Klient, który w przyszłości będzie rozwijać i utrzymywać projekt preferuje język C#. Członkowie zespołu mają szeroką wiedzę i duże doświadczenie ze środowiskiem .NET. Użycie innego języka niż C# nie przyniosłoby wymiernych korzyści, a podstawowa potrzeba uruchomienia kodu studenta implikuje konieczność użycia środowiska .NET na pewnym etapie działania rozwiązania. Konteneryzacja pozwala w bezpieczny sposób uruchamiać obcy kod, znacząco redukując ryzyko związane z próbami wywołania kodu złośliwego.
Dystrybucja	Paczka NuGet	Plik biblioteki (`.dll`)	Prosty sposób dystrybucji biblioteki poprzez natywny menedżer pakietów. Pozwala na łatwe aktualizowanie paczki bez konieczności wysyłania żadnych plików studentom. Ponadto integracja poprzez system CI/CD (pkt niżej) całkowicie przejmuje obowiązek wdrożenia nowej wersji od programisty.
CI/CD	GitHub Actions	Ręczne uruchamianie	Automatyczna kompilacja rozwiązania, przeprowadzanie testów poprawności i tworzenie kontenerów Docker, a także wypychanie gotowych artefaktów do repozytoriów DockerHub i NuGet. Brak automatyzacji tego procesu nie przyniósłby żadnych korzyści. Jest to standard branżowy.
Ocenianie i eksport wyników	GitHub Actions (lub odpowiednik)	Ręczne uruchamianie przez prowadzącego	Automatyczne uruchamianie systemu oceniającego przy każdym wypchnięciu kodu do repozytorium przez studenta. Wysyłanie raportów z wynikami bezpośrednio do dedykowanego repozytorium w celach archiwizacji, wystawiania ocen studentom i prezentacji tych danych na stronie internetowej przedmiotu ASD2. Alternatywą jest ręczne wprowadzanie tych danych do systemu przez prowadzącego, co nie przynosi żadnych korzyści, jest żmudne i może powodować błędy.

3 Specyfikacja interfejsu fasady testowej

Fasada testowa opiera się na wzorcu AAA (*Arrange-Act-Assert*) oraz deklaratywnym podejściu do definiowania zestawów testowych. Prowadzący tworzy zestawy testów dla konkretnego zadania, definiując jedną klasę dziedziczącą po klasie zestawu testowego i oznaczając ją odpowiednimi atrybutami. Celem fasady jest zminimalizowanie niepotrzebnego, powtarzalnego kodu oraz zapewnienie pełnej automatyzacji procesu weryfikacji i oceny kodu.

3.1 Kluczowe komponenty fasady

W celu zapewnienia jak najlepszej ergonomii pracy z systemem, kluczowym zadaniem jest zdefiniowanie fasady, która:

- w maksymalnym stopniu ogranicza potrzebę pisania powtarzalnego kodu,
- wykorzystuje mechanizmy wspierające tworzenie kodu w IDE, np. automatyczne tworzenie szkieletu klasy na podstawie definicji jej klasy bazowej,
- wykazuje podobieństwo do fasady aktualnie używanego systemu.

Zaproponowano fasadę, której przykładowe użycie zaprezentowano jako kod źródłowy 1.

```
[TestSuite]
[Containerized] // (2)
[StudentSolution(typeof(Student1Solution))] // (1)
public partial class Lab01Evaluation : TestSuite
{
    [TestCase]
    [InstructionLimit(5_000_000)] // (2)
    [TimeLimitMs(30_000)] // (2)
    [MemoryLimitMb(256)] // (2)
    public class Case1 : TestCase // (3)
    {
        private int _liczba = 3;

        public override void Arrange()
        {
            _liczba = 5;
        }

        public override void Act()
        {
            _liczba *= _liczba;
        }

        public override void Assert()
        {
            IsTrue(_liczba > 0);
            IsTrue(_liczba == 25);
        }
    }

    // kolejne przypadki testowe...
}
```

Kod źródłowy 1: Przykładowe użycie fasady do definicji zestawu testowego.

W treści kodu źródłowego 1 znajdują się fragmenty oznaczone numerami. Poniżej opisano je dokładniej.

1. Atrybut wiążący zestaw testów z konkretnym rozwiązaniem studenta. Jest ono pewną klasą zawierającą implementacje funkcji, które są testowane. Z założenia, pusta wersja takiej klasy jest dostarczana studentowi na początku zajęć. Zadaniem studenta jest jej uzupełnienie.
2. Atrybuty umożliwiające podstawową konfigurację testu. Możliwe będą takie parametry jak możliwość konteneryzacji testu lub maksymalna ilość instrukcji CIL lub limit czasu wykonania.

3. Każdy przypadek testowy stanowi osobną, zagnieżdżoną klasę dziedziczącą po klasie przypadku testowego. Wymusza ona podział logiki na trzy czytelne fazy:
    - Przygotowanie (*Arrange*): Przygotowanie testu poprzez stworzenie wymaganych struktur, klas, obiektów potrzebnych do przeprowadzenia testu.
    - Wykonanie akcji (*Act*): Wywołanie właściwej metody z kodu studenta.
    - Weryfikacja (*Assert*): Weryfikacja za pomocą dedykowanych metod asercji, sprawdzających nie tylko poprawność wyniku, ale również efekty uboczne (np. brak spójności grafu itp.). Dzięki modyfikowalnej funkcji `Assert()` prowadzący nie będzie ograniczony do sprawdzenia jedynie wyniku, lecz będzie w stanie przetestować dowolny warunek.

3.2 Automatyzacja i generowanie kodu zestawów testowych

Aby prowadzący nie musiał ręcznie pisać kodu integrującego klasę testową z silnikiem wykonawczym TestExecutor ani tworzyć powtarzalnych metod uruchamiających w środowisku MSTest, platforma wykorzystuje przyrostowy generator kodu kompilatora Roslyn. Generator działa całkowicie statycznie w trakcie kompilacji projektu testowego i wykonuje następujące zadania:

3.2.1 Filtrowanie i analiza składniowa

W etapie inicjalizacji generator skanuje drzewo składniowe C# i wyszukuje deklaracje klas (`ClassDeclarationSyntax`), które spełniają następujące kryteria architektoniczne:

- Klasa musi być publiczna, nieabstrakcyjna i zadeklarowana na poziomie przestrzeni nazw.
- Klasa musi bezpośrednio lub pośrednio dziedziczyć po typie bazowym TestSuite.

Dla każdej prawidłowej klasy generator odczytuje atrybut `[StudentSolution]`, używając modelu semantycznego Roslyn do pobrania symbolu typu rozwiązania studenta. Następnie wyciąga surowy kod źródłowy drzewa składniowego zadania studenta oraz kod samej klasy testowej. Dodatkowo przeszukuje zagnieżdżone typy w celu zidentyfikowania wszystkich klas przypadków testowych dziedziczących po TestCase.

3.2.2 Generowanie kodu wyjściowego

Dla każdego odnalezionego zestawu testowego generator automatycznie tworzy w pamięci kompilatora plik kodowy (`.g.cs`), zawierający dwie kluczowe struktury:

1. Rozszerzenie klasy zestawu testowego, które uzupełnia zadeklarowaną przez prowadzącego klasę o przesłonięte metody dostępowe zwracające surowy kod źródłowy w postaci surowych literałów ciągów znaków

```
public partial class Lab03Evaluation
{
    public override string StudentSolutionSourceCode() => """..."""";
    public override string TestSuiteSourceCode() => """..."""";
}
```

Kod źródłowy 2: Wygenerowane rozszerzenie klasy zestawu testowego o metody zwracające kod źródłowy rozwiązania studenta i zestawu testów.

   Pozwala to orkiestratorowi TestExecutor na bezpośredni dostęp do czystych tekstów źródłowych w celu ich późniejszej analizy statycznej i dynamicznej kompilacji Roslyn bez konieczności odczytywania plików z dysku w trakcie wykonywania

2. Wydzielona klasa uruchomieniowa MSTest: Dla zestawu testowego generowana jest towarzysząca klasa z przyrostkiem _Tests oznaczona atrybutem `[TestClass]`. Dla każdego zidentyfikowanego przypadku testowego (`TestCase`) generator generuje osobną asynchroniczną metodę testową `[TestMethod]`:

```
[TestClass]
public partial class Lab03Evaluation_Tests
{
    private static Lab03Evaluation _suite = new Lab03Evaluation();
    private static TestExecutor _executor = new TestExecutor(_suite);
```

```
    [TestMethod]
    public async Task Case0_Reverse_Gesty()
    {
        _executor.BeginExecution();
        var result = await _executor.GetResult(0);
        if (result.IsT1)
        {
            var failure = result.AsT1;
            Assert.Fail(failure.Message ?? "Test failed");
        }
    }
    // kolejne metody dla każdego przypadku testowego
}
```

Kod źródłowy 3: Wygenerowana klasa uruchomieniowa MSTest z metodą testową dla pierwszego przypadku testowego.

Dzięki tak zaprojektowanemu generatorowi prowadzący definiuje wyłącznie zwięzłą klasę deklaratywną AAA, a cały mechanizm orkiestracji, integracji z narzędziami uruchamiania testów w IDE (Visual Studio / JetBrains Rider) oraz asynchronicznego oczekiwania na wyniki z TestExecutor generowany jest automatycznie.

3.3 Przepływ tworzenia zadań przez prowadzącego

Proces weryfikacji i przygotowania zadań przebiega według następujących kroków:

1. Prowadzący definiuje interfejs dla studenta. Student dostaje wyłącznie czyste nagłówki metod i klasę z wymaganą sygnaturą.
2. Prowadzący tworzy zróżnicowane przypadki testowe, które mają na celu weryfikację poprawności kodu studenta. Dla każdego testu definiowana jest klasa przypadku testowego zawierająca dedykowaną logikę sprawdzającą.
3. Studenci implementują rozwiązania zadania. Dzięki testom zdefiniowanym przez prowadzącego mogą samodzielnie i na bieżąco weryfikować poprawność swojego kodu w trakcie pracy.
4. Na koniec studenci dostają raport błędów zwróconych przez sprawdzarkę. W przypadku niepowodzenia fasada zwraca studentowi czytelny komunikat diagnostyczny informujący o niepowodzeniu kompilacji, przekroczonej dopuszczalnej ilości operacji lub niespełnieniu asercji testowej.